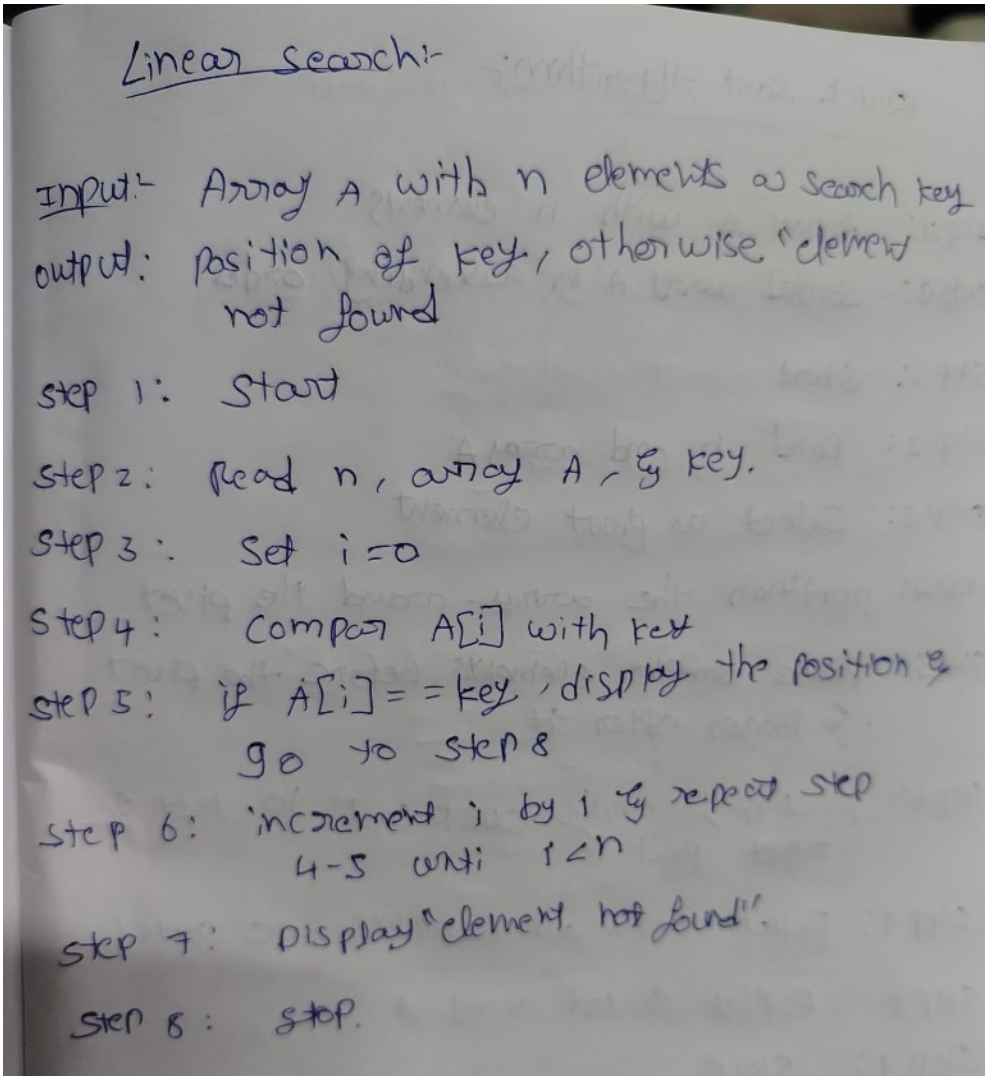
PRACTICAL-2

AIM: To implement and analyze the time complexity of Linear Search and Binary Search algorithms, and compare their searching efficiency for different input sizes.

LINEAR SEARCH:

ALGORITHM:



PROGRAM/CODE:

```
#include <iostream>
using namespace std;

int main()
{
   int size, arr[100], key, i;

   cout << "Enter the size of the array: ";
   cin >> size;

   cout << "Enter " << size << " elements: ";
   for (i = 0; i < size; i++)
   {
      cin >> arr[i];
   }

   cout << "Enter the element to be searched: ";
   cin >> key;

   i = 0;

   for (i = 0; i < size; i++)
   {
      if (arr[i] == key)
      {
         cout << "Element found at position " << i + 1;
         break;
      }
   }

   if (i == size)
   {
      cout << "Element not found";
   }

   return 0;
}
```

OUTPUT:

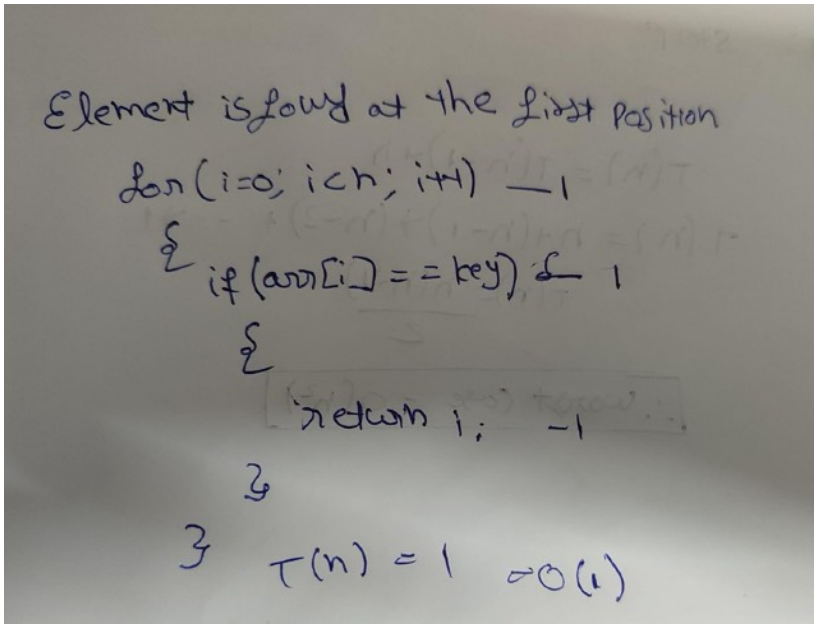
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

nandhagopal@NGKs-MacBook-Air c++ % /Users/nandhagopal/Documents/CODING/c++/linearsearch
Enter the size of the array: 4
Enter 4 elements: 23
55
11
88
Enter the element to be searched: 23
Element found at position 1
nandhagopal@NGKs-MacBook-Air c++ %
```

TIME COMPLEXITY:

1.BEST CASE:

The best case occurs when the search element is found at the first position.

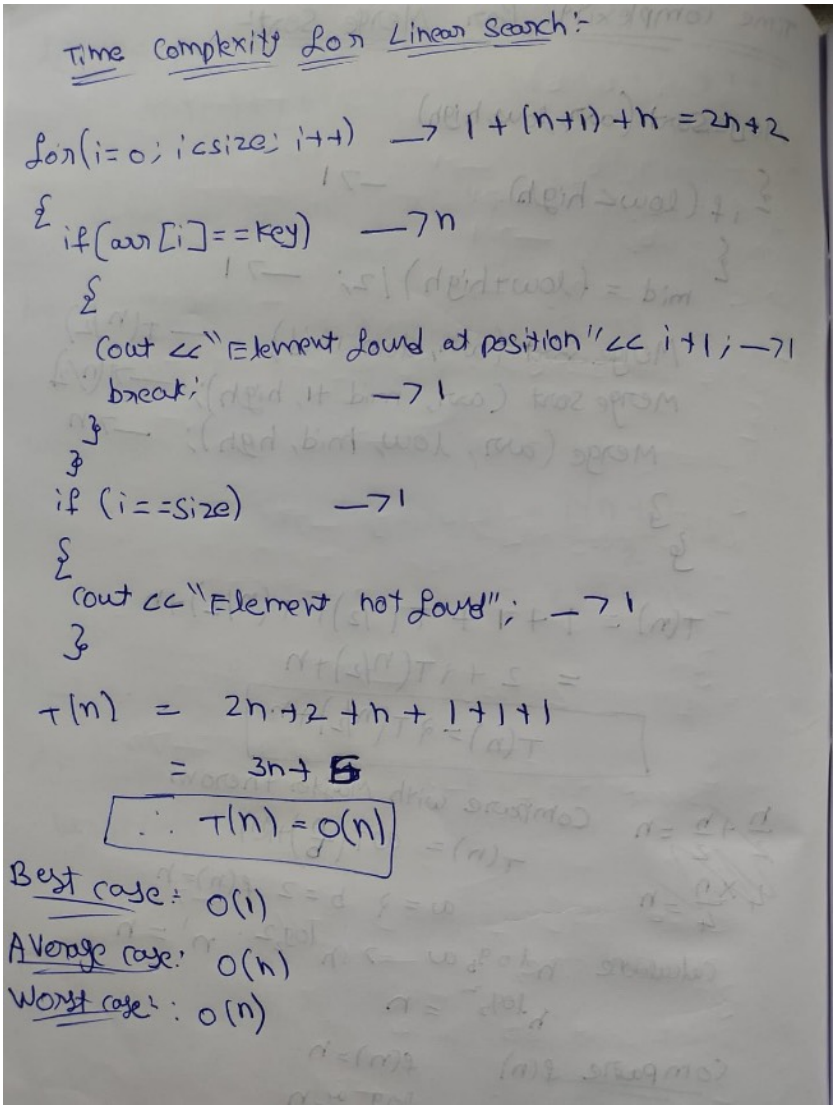


2.AVERAGE CASE:

The average case occurs when the element is found somewhere in the middle of the array.

3.WORST CASE:

The worst case occurs when the element is at the last position or is not present in the array.



Time Complexity For Linear Search:-

```
for(i=0; i<size; i++)   -> 1 + (n+1) + n = 2n+2
{
  if(arr[i]==key)       -> n
  {
    cout << "Element found at position" << i+1;   -> 1
    break;              -> 1
  }
}
if (i==size)            -> 1
{
  cout << "Element not found";   -> 1
}
```

$T(n) = 2n+2+n+1+1+1$

$= 3n+5$

$\therefore T(n) = O(n)$

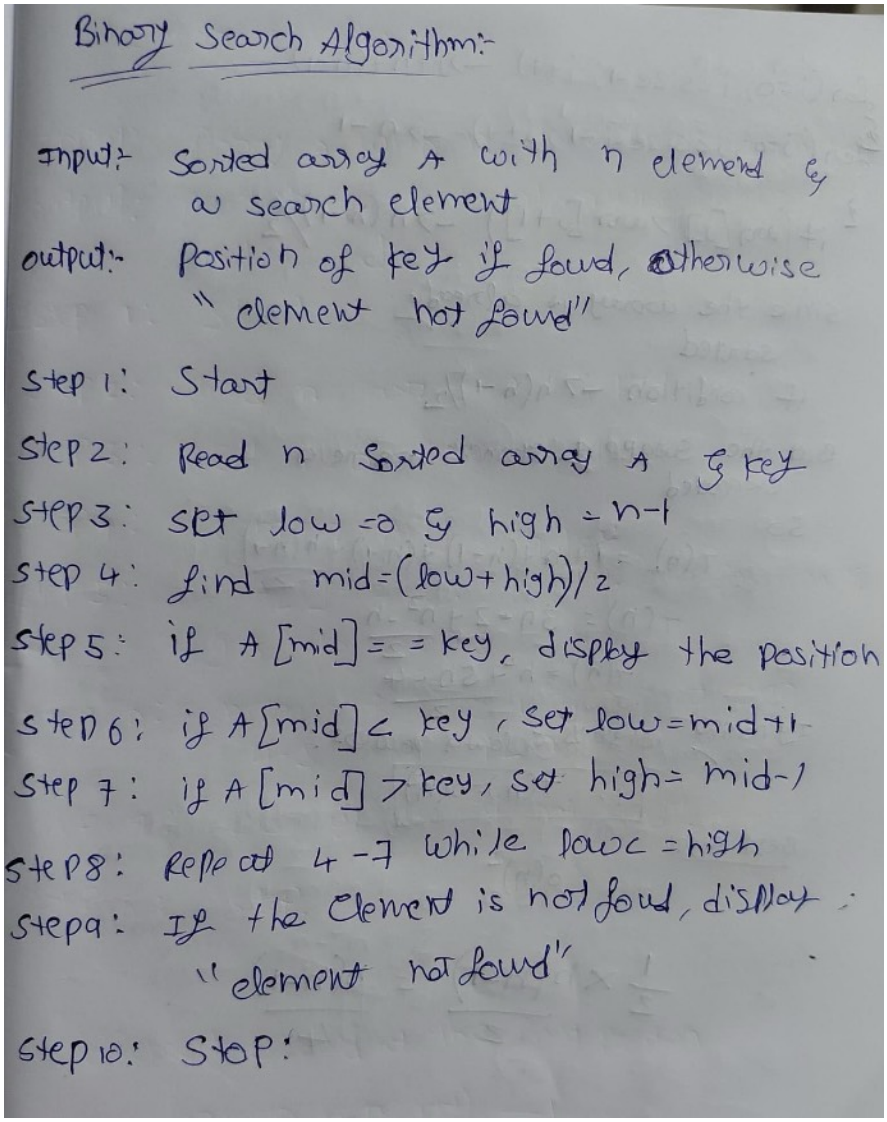
Best case: $O(1)$

Average case: $O(n)$

Worst case: $O(n)$

BINARY SEARCH:

ALGORITHM:



PROGRAM/CODE:

```
#include <iostream>
using namespace std;

int BinarySearch(int arr[], int size, int search)
{
   int low = 0;
   int high = size - 1;

   while (low <= high)
   {
      int mid = (low + high) / 2;

      if (search == arr[mid])
      {
         return mid;
```

```
        }
        else if (search < arr[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    return -1;
}

int main()
{
    int size, arr[100], search, result;

    cout << "Enter the size of the array: ";
    cin >> size;

    cout << "Enter " << size << " elements in sorted order: ";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }

    cout << "Enter the element to be searched: ";
    cin >> search;

    result = BinarySearch(arr, size, search);

    if (result != -1)
    {
        cout << "Element found at position " << result + 1;
    }
    else
    {
        cout << "Element not found";
    }

    return 0;
}
```

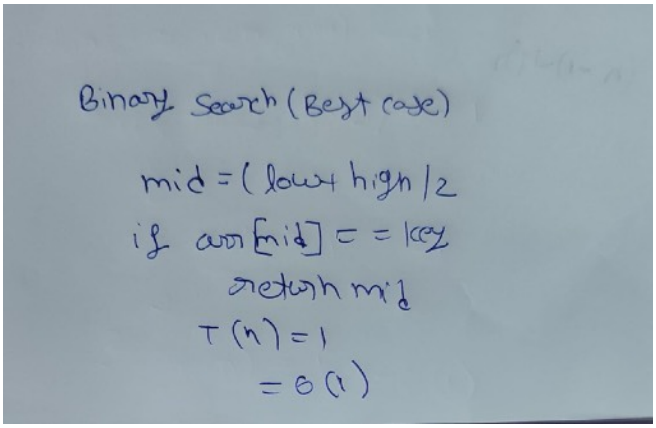
OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

nandhagopal@NGKs-MacBook-Air c++ % /Users/nandhagopal/Documents/CODING/c++/binarysearch
Enter the size of the array: 4
Enter 4 elements in sorted order: 22
11
44
77
Enter the element to be searched: 44
Element found at position 3
nandhagopal@NGKs-MacBook-Air c++ %
```

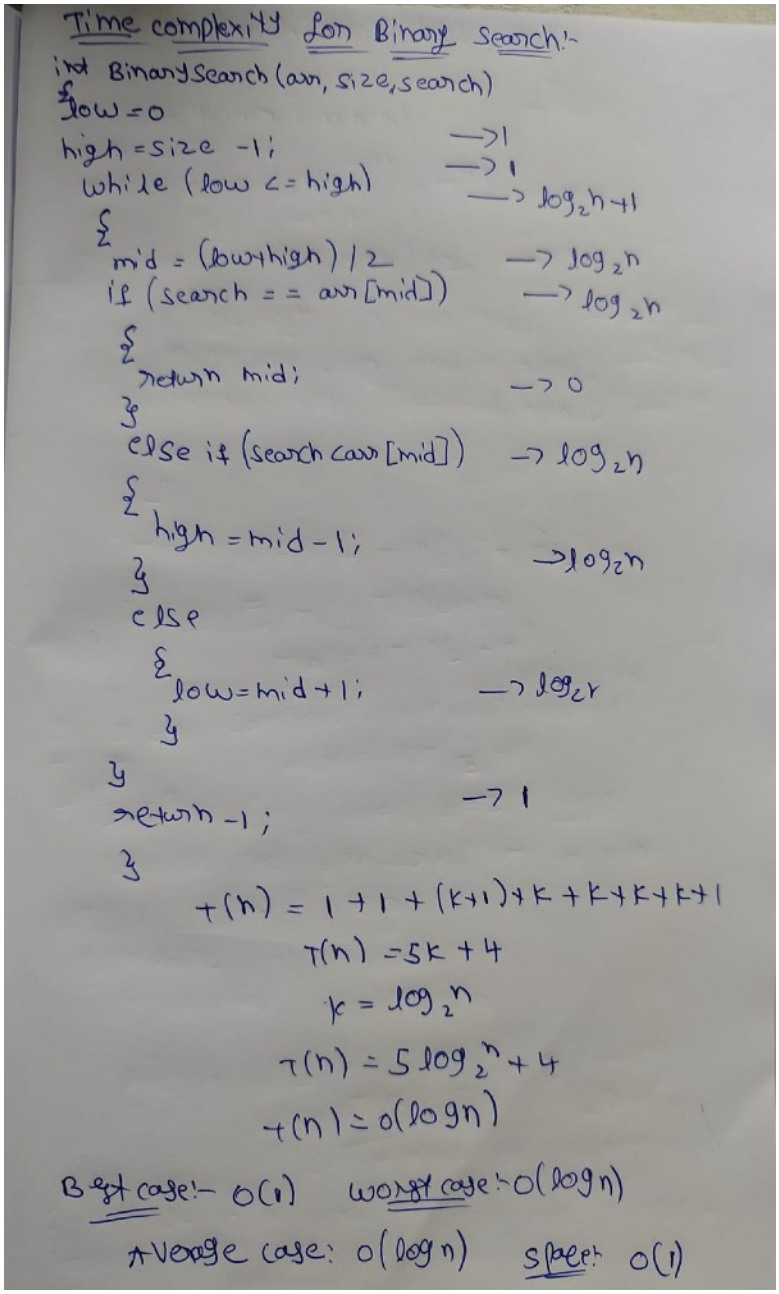
TIME COMPLEXITY:

1.BEST CASE:
The best case occurs when the search element is found at the middle position in the first comparison.



2.AVERAGE CASE:
The average case occurs when the element is found after several divisions of the search space.

3.WORST CASE:
The worst case occurs when the element is found after maximum divisions or is not present in the array.



CONCLUSION:

Thus, Linear Search and Binary Search were successfully implemented and their time complexities were analyzed. It was observed that Linear Search has a time complexity of O(n), while Binary Search has O(log n), making Binary Search more efficient for searching in a sorted array.